KHora: A Model Checker for Knowing-How Logics

Rens Doodeman, Ziqi Wang, and Zixuan Chen

Sunday 29th March, 2026

Abstract

We present KHora, a model checking tool for both the basic knowing-how logic $\mathcal{L}_{Kh}$ and the uncertainty-based knowing-how logic with regularity constraints reg-$\mathcal{L}^{U}_{Kh}$ in Haskell. The implementation of $\mathcal{L}_{Kh}$ uses an explicit bounded search over plans, while the implementation of reg-$\mathcal{L}^{U}_{Kh}$ is based on reducing the model-checking conditions to graph reachability checks. To support experimentation, we also provide parsers for both logics, a collection of `QuickCheck` tests for basic semantic and implementation-level properties, and a lightweight web interface for better user experience.

Contents

1 Introduction **3**

2 Basic Knowing How Logic $\mathcal{L}_{Kh}$ **3**

2.1 Preliminaries 3

2.2 Haskell Representation 4

2.3 Model checker in Haskell 5

2.4 Parsing for $\mathcal{L}_{Kh}$ 6

2.5 Random Generation for $\mathcal{L}_{Kh}$ 7

2.5.1 Model Generation with Parameters 7

2.5.2 Arbitrary Instances for QuickCheck 7

3 Uncertainty-based Knowing-how Logic with Regularity Constraints reg-$\mathcal{L}^{U}_{Kh}$ **8**

3.1 Preliminaries 8

3.2 Haskell Representation 10

3.3 Model checker in Haskell 11
3.4 Parsing for reg-$\mathcal{L}_{Kh}^{U}$ 15
3.5 Random Generation for reg-$\mathcal{L}_{Kh}^{U}$ 16
3.5.1 Model Generation with Parameters 16
3.5.2 Arbitrary Instances for QuickCheck 17

4 Testing and Validation **17**
4.1 Tests for $\mathcal{L}_{Kh}$ 17
4.2 Tests for reg-$\mathcal{L}_{Kh}^{U}$ 18

5 Web Interface **19**

6 Conclusion **19**

Bibliography **20**

1 Introduction

Knowing-how logics study an agent's ability to achieve goals by suitable action. This makes them particularly relevant to artificial intelligence, especially to planning, where the main question is whether a desired objective can be guaranteed by some plan.

The simplest knowing-how logic, $\mathcal{L}_{Kh}$, was first introduced in [3]. This logic captures an agent's ability to guarantee the achievement of a goal under a given condition. In this sense, the notion is somewhat idealized: even if an agent manages to achieve something under a condition, this does not necessarily mean that the agent genuinely knows how to do it. For example, winning a lottery does not imply knowing how to win it. In such cases, uncertainty becomes essential.

To address this, one may move to the uncertainty-based knowing-how logic $\mathcal{L}^U_{Kh}$ [1], which supports multi-agent reasoning. In this logic, an agent may have several indistinguishable plans for achieving the same purpose. This adds an epistemic feature to the basic knowing-how logic: plans are grouped into equivalence classes representing the agent's uncertainty about which plan is being followed. Finally, the uncertainty-based logic with regularity constraints reg-$\mathcal{L}^U_{Kh}$ [2] connects this approach with formal language theory by representing such equivalence classes using finite automata.

In this work, we present KHora, a model-checking tool for both $\mathcal{L}_{Kh}$ and reg-$\mathcal{L}^U_{Kh}$ in Haskell. For $\mathcal{L}_{Kh}$, the tool implements an explicit bounded search over plans; for reg-$\mathcal{L}^U_{Kh}$, it realises the algorithms of [2] in an executable setting. In addition to the core model checkers, KHora includes parsers for both logics, random generators for models and formulas, a collection of tests based on `QuickCheck` and `HSpec`, and a lightweight web interface that visualizes the generation of formulas and models.

The repository is available on GitHub at:
`https://github.com/Knowing-How-Logics/haskell-knowing-how-logics`.

2 Basic Knowing How Logic $\mathcal{L}_{Kh}$

In this section we model the language of $\mathcal{L}_{Kh}$ introduced in [3]. This captures the basic idea of an agent knowing how to achieve a goal under certain condition. We implement an explicit model-checker for this logic with a bounded plan depth, along with `QuickCheck` generators for random testing of the semantics.

2.1 Preliminaries

Definition 2.1. (Syntax) Given a set of proposition letters $Prop$, the language $\mathcal{L}_{Kh}$ is defined by

$$\varphi ::= \top \mid p \mid \neg\varphi \mid \varphi \wedge \varphi \mid Kh(\varphi, \varphi),$$

where $p \in Prop$.

Definition 2.2 (LTS)**.** A *labelled transition system* (LTS) is a tuple $\mathcal{M} = (S, (R_a)_{a \in Act}, V)$, where:

- S is a non-empty set of *states*,
- $(R_a)_{a \in Act}$ is a family of *binary relations* on S, each labelled by an *action* $a \in Act$,
- $V : S \to 2^{Prop}$ is a *valuation function*.

We write $s \xrightarrow{a} t$ if $(s,t) \in R_a$. For a plan $\sigma = a_1 \dots a_n \in Act^*$, we write $s \xrightarrow{\sigma} t$ if there exist $s_1, \dots, s_{n-1}$ such that $s \xrightarrow{a_1} s_1 \xrightarrow{a_2} \cdots \xrightarrow{a_n} t$. A plan $\sigma = a_1 \dots a_n$ is *strongly executable* at s iff for every $0 \leq k < n$ and every t such that $s \xrightarrow{\sigma_k} t$, the state t has at least one a_{k+1}-successor.

Definition 2.3 (Semantics)**.** Let $\mathcal{M}$ be an LTS, and s be a state.

$\mathcal{M}, s \models \top$	*iff*	*always*
$\mathcal{M}, s \models p$	*iff*	$p \in V(s)$
$\mathcal{M}, s \models \neg\varphi$	*iff*	$\mathcal{M}, s \not\models \varphi$
$\mathcal{M}, s \models \varphi \wedge \psi$	*iff*	$\mathcal{M}, s \models \varphi$ and $\mathcal{M}, s \models \psi$
$\mathcal{M}, s \models Kh(\psi, \varphi)$	*iff*	there exists a $\sigma \in Act^*$ such that for all s' such that $\mathcal{M}, s' \models \psi$: σ is strongly executable at s' and for all t such that $s' \xrightarrow{\sigma} t$, $\mathcal{M}, t \models \varphi$

The $Kh(\psi, \varphi)$ can be interpreted as "the agent knows how to achieve φ given ψ". Notice that the Kh is a global modality. Namely, either it holds for all states, or none of them.

2.2 Haskell Representation

The syntax is modelled following Definition 2.1.

```
type Proposition = Int

data Form = P Proposition | Neg Form | Conj Form Form | KH Form Form | T
    deriving (Eq, Show, Ord)
```

Following Definition 2.2, we model the LTS as follows. When creating a LTS, consider that states is a non-empty list and therefore must use the :| constructor, i.e. (n :| [n+1..]).

```
type Action = Int
type Plan = [Action]
type State = Int
type Valuation = [(State, [Proposition])]

data AbilityMap = LTS {
    states :: NonEmpty State,
    transitions :: Relations,
    valuation :: Valuation
} deriving(Show)
```

The relations and strong executability in Definition 2.2 are implemented below. Both of these are binary relations on states. Therefore, in our implementation, we use the same Haskell type `Rel` to represent them, since both can be understood as sets of pairs of states.

To represent the family of atomic relations $R = (R_a)_a$, we index relations by actions. This gives rise to the type `Relations`, corresponding to the collection of action-labelled transitions in the literature.

We do not model R_σ explicitly as a separate data structure. Instead, we only implement the operations needed to reason about plan execution and strong executability.

For this purpose, we define the following helper functions:

- `image`, which computes the image of a state under a relation, i.e. it collects all successor states of a given state;
- `r_a`, which takes an action a and the family of relations R, and returns the indexed relation R_a;
- `executePlan`, which computes all possible end states obtained after executing a plan from a given initial state;
- `stronglyExecutableAt`, which determines whether a plan is strongly executable at a given state.

```
-- Both $R_a$ and $R_\sigma$ share the same type.
-- Here we model the whole set of relations, without classification.
type Rel = [(State, State)]

-- The family of action-indexed relations (R_a)_a
type Relations = [(Action, Rel)]

-- Image of a state under R_a
image :: Rel -> State -> [State]
image r u = [v | (x, v) <- r, x == u]

-- Given (R_a)_a and an action x, return R_x
r_a :: Relations -> Action -> Rel
r_a rs a = fromMaybe [] (lookup a rs)

-- Execute a plan from a given start state
-- This corresponds to R_sigma(u)
executePlan :: Relations -> State -> Plan -> [State]
executePlan _  u []       = [u]
executePlan rs u (a:sigma) =
    nub (concat [ executePlan rs v sigma | v <- image (r_a rs a) u ])

-- Plan is SE or not at a given state
stronglyExecutableAt :: Relations -> State -> Plan -> Bool
stronglyExecutableAt _  _ []       = True
stronglyExecutableAt rs u (a:sigma) =
    let next = image (r_a rs a) u
    in  not (null next) &&
        all (\v -> stronglyExecutableAt rs v sigma) next
```

2.3 Model checker in Haskell

We now implement the semantics in Definition 2.3. Note that `findPlans` enumerates all plans up to a fixed depth (currently 5). This is a bounded approximation: if a witness plan longer than the bound exists, the checker will not find it. A complete decision procedure would require another PSPACE-Complete algorithm in Theorem 1 from [2].

```
-- Given an LTS and formula, returns the states that satisfy said formula
statesSatisifying :: AbilityMap -> Form -> [State]
statesSatisifying m f = [s | s <- toList (states m), isTrue (m, s) f]

-- Given an LTS, find all plans.
findPlans :: AbilityMap -> [Plan]
```

```haskell
findPlans m = nub (concatMap (plansFrom depth) (toList (states m)))
  where
    -- For now we limit the depth to avoid infinite loops.
    depth = 5

    -- Flatten transitions
    edges :: [(Action,(State,State))]
    edges = [ (a,(u,v))| (a,rel) <- transitions m, (u,v) <- rel ]

    -- Generate all plans from a given state up to our specified depth
    plansFrom :: Int -> State -> [Plan]
    plansFrom 0 _ = []
    plansFrom d s =
        [ [a] | (a,(x,_)) <- edges, x == s ] -- single action plans
        ++ [ a:p | (a,(x,y)) <- edges, x == s, p <- plansFrom (d-1) y ]

isTrue :: (AbilityMap, State) -> Form -> Bool
```

```haskell
-- KH is NOT local; its truth does not depend on the state at which it is evaluated.
-- KH either holds at all states, or none of them.
isTrue (m, _) (KH f g) =
    any (\a ->
        all (\s ->
            stronglyExecutableAt rs s a
            && all (\t -> isTrue (m,t) g) (executePlan rs s a)
        ) statesF
    ) candidatePlans
  where
    rs = transitions m
    statesF = statesSatisifying m f
    candidatePlans = findPlans m

-- Infix alias for the satisfaction relation
(|=) :: (AbilityMap, State) -> Form -> Bool
(|=) = isTrue
```

2.4 Parsing for $\mathcal{L}_{Kh}$

Formulas may be created in `ghci` using `parseForm`. We omit most of the code here. The following inputs are accepted:

- `p` or `P` followed by an integer n returns `P n`
- `T` returns `T`
- `!p` returns `Neg p`
- `p & q` returns `Conj p q`
- `KH p q` returns `KH p q`
- `p -> q` returns `Neg (Conj p (Neg q))` (as an abbreviation)
- `p v q` or `p V q` returns `Neg (Conj (Neg p) (Neg q))` (as an abbreviation)

```haskell
parseForm :: String -> Either ParseError Form
parseForm = parse (pForm <* eof) "input"

evalForm :: (AbilityMap, State) -> String -> Bool
evalForm (m, s) str =
    case parseForm str of
        Right f -> isTrue (m, s) f
        Left _  -> error "Invalid formula"
```

2.5 Random Generation for $\mathcal{L}_{Kh}$

2.5.1 Model Generation with Parameters

To facilitate testing with models of specific sizes, we implement a function that generates an *AbilityMap* from a fixed number of states, propositions, and actions. We omit the code here.

The generated model includes:

- a finite set of states $\{1, \ldots, n\}$,
- a set of actions $\{1, \ldots, k\}$,
- a valuation assigning each state a random subset of propositions $\{1, \ldots, m\}$,
- a family of action-labelled transitions, each generated by randomly sampling pairs of states.

It is possible to generate concrete models and test formulas interactively in `ghci`. For instance:

```
ghci> m <- generate (generateLTS 5 3 2)
ghci> m
LTS {states = 1 :| [2,3,4,5],
transitions = [(1,[(4,5),(4,2),(4,3),(5,1),(1,4),(2,2),
(3,1),(4,4),(2,3)]), (2,[(4,3),(4,5),(2,5),(4,2),(5,4),(1,5)])],
valuation = [(1,[1,3]),(2,[2,3]),(3,[1,2,3]),(4,[2,3]),(5,[2])]}

-- Evaluate a formula at a state
ghci> isTrue (m, 1) (KH (P 1) (P 2))
True

-- Using the parser
ghci> evalForm (m, 1) "KH p1 p2"
True
```

In the example above, `generateLTS 5 3 2` produces a random LTS with five states, three proposition letters, and two actions. The function `isTrue` can then be used to evaluate formulas directly, while `evalForm` allows testing formulas given as strings.

2.5.2 Arbitrary Instances for QuickCheck

Finally, for this section we define the instances of Arbitrary for `Form` and `Arbitrary` respectively. We omit the code here.

It is possible to try out semantics by running `stack ghci`. For example:

```
ghci> generate (arbitrary:: Gen AbilityMap)
LTS {states = 1 :| [2,3,4,5,6], ...
```

```
ghci> m = LTS {states = 1 :| [2,3,4,5,6], ...
ghci> isTrue (m, 2) (KH (P 4) (P 1))
False
ghci> evalForm (m, 2) "KH (p4 ^ !p2) p3"
True
```

3 Uncertainty-based Knowing-how Logic with Regularity Constraints $reg\text{-}\mathcal{L}^U_{Kh}$

In the framework of *basic knowing how* we introduced above, an agent possesses the ability to achieve the goal φ given ψ if and only if he has a plan that is fail-proof, meaning that each partial execution must be completable. In scenarios where the agent lacks this ability, it is only because a sequence of actions cannot be generated due to certain environmental constraints. However, another scenario may also occur: the agent does not know which plan is adequate for the situation. All he can do is blindly apply a plan he thought might work, which may or may not be successful. Such *indistinguishability* among plans establishes an epistemic relation of *knowing that*.

We now introduce a new logic of *knowing how* for a multi-agent setting defined in [2], extending the LTS with a notion of epistemic indistinguishability between plans. The uncertainty of an agent is encoded by equivalence classes of plans, and each such class is represented by a finite automaton. More precisely, each equivalence class of indistinguishable plans is represented by the language recognised by a finite automaton.

In Theorem 2 from [2], it is shown that the model checking problem for this logic is in PTIME. The proof relies on reducing both conditions of the Kh_i-modality to graph reachability checks. In this section, we provide a concrete implementation of this algorithm in Haskell, closely following the constructions from the paper.

3.1 Preliminaries

Definition 3.1 (Syntax)**.** Given a set of proposition letters $Prop$ and a set of agents Agt, where $p \in Prop$ and $i \in Agt$, the language $reg\text{-}\mathcal{L}^U_{Kh}$ is defined by

$$\varphi := p \mid \neg\varphi \mid \varphi \vee \varphi \mid Kh_i(\psi, \varphi).$$

We first introduce the automata used to represent classes of plans.

Definition 3.2 (Automaton)**.** A non-deterministic finite automaton is a tuple $\mathcal{A} = (Q, Act, \delta, I, F)$, where:

- Q is a finite set of states,
- Act is a finite alphabet of actions,
- $\delta : Q \times Act \to 2^Q$ is a transition function,
- $I \subseteq Q$ is the set of initial states,

- $F \subseteq Q$ is the set of accepting (final) states.

To connect an automaton with the underlying transition system, we use the following product digraph.

Definition 3.3 (Digraph)**.** Given a reg-LTS^U $\mathcal{S}$ and an automaton $\mathcal{A} = (Q, Act, \delta, I, F)$, we define a digraph $G = (V, E)$ as follows:

- $V = Q \times S$, representing the combined states of the automaton and the underlying transition system;
- E is the set of edges such that $((q, t), (q', t')) \in E$ if and only if there exists an action $a \in Act$ such that:
  1. $q \xrightarrow{a} q'$ in $\mathcal{A}$;
  2. $(t, t') \in R_a$.

This construction synchronizes transitions of the automaton with transitions of the LTS under the same action, allowing us to track which runs of the automaton are realizable in the model.

We next recall the relational notions needed for the semantics.

Definition 3.4. For $\pi \subseteq Act^*$, $u \in S$, and $U \subseteq S$, define

$$R_\pi := \bigcup_{\sigma \in \pi} R_\sigma, \qquad R_\pi(u) := \bigcup_{\sigma \in \pi} R_\sigma(u), \qquad R_\pi(U) := \bigcup_{u \in U} R_\pi(u).$$

A set of plans $\pi \subseteq Act^*$ is *strongly executable* at $u \in S$ iff every plan $\sigma \in \pi$ is strongly executable at u. This gives rise to the definition $SE(\pi) := \bigcap_{\sigma \in \pi} SE(\sigma)$, which collects all states at which every plan in π is strongly executable.

These notions can be lifted from plan sets to automata via their accepted languages.

Definition 3.5. In terms of automata, $\mathrm{SE}(L(\mathcal{A})) := \bigcap_{\sigma \in L(\mathcal{A})} \mathrm{SE}(\sigma)$ collects all states at which every plan in $L(\mathcal{A})$ is strongly executable, and $R_{L(\mathcal{A})} := \bigcup_{\sigma \in L(\mathcal{A})} R_\sigma$ collects all transitions realisable by some plan in $L(\mathcal{A})$.

We now define reg-LTS^U models.

Definition 3.6. A reg-LTS^U is a tuple $\mathcal{S} = (S, (R_a)_{a \in Act}, (U_i)_{i \in Agt}, V)$, where the elements of U_i associated with an agent i are finite automata, namely,

$$\emptyset \neq U_i = \{\mathcal{A}_1, \mathcal{A}_2, \ldots\}.$$

Here, $L(\mathcal{A}_i)$ denotes the language accepted by $\mathcal{A}_i$. Moreover, for every $\mathcal{A}_m, \mathcal{A}_n \in U_i$ such that $m \neq n$, we have

$$L(\mathcal{A}_m) \cap L(\mathcal{A}_n) = \emptyset.$$

Finally, we can state the semantics of the language.

Definition 3.7 (Semantics). Let $\mathcal{S} = (S, (R_a)_{a \in Act}, (U_i)_{i \in Agt}, V)$ be a finite reg-LTS^U, and $s \in S$. The satisfaction relation $\models$ is defined as:

$\mathcal{S}, s \Vdash p$	*iff*	$p \in V(s)$
$\mathcal{S}, s \Vdash \neg\varphi$	*iff*	$\mathcal{S}, s \nVdash \varphi$
$\mathcal{S}, s \Vdash \psi \vee \varphi$	*iff*	$\mathcal{S}, s \Vdash \psi$ or $\mathcal{S}, s \Vdash \varphi$
$\mathcal{S}, s \Vdash Kh_i(\psi, \varphi)$	*iff*	there is a finite automaton $\mathcal{A} \in U_i$, such that (1) $[\![\psi]\!]^{\mathcal{S}} \subseteq SE(L(\mathcal{A}))$ and (2) for every $t \in [\![\psi]\!]^{\mathcal{S}}$, $R_{L(\mathcal{A})}(t) \subseteq [\![\varphi]\!]^{\mathcal{S}}$,

where $[\![\psi]\!]^{\mathcal{S}} := \{s \in S \mid \mathcal{S}, s \Vdash \psi\}$ is the set of all states satisfying ψ.

The $Kh_i(\psi, \varphi)$ can be interpreted as "when ψ is the case, the agent i knows how to make φ true". Kh_i is also a global modality.

3.2 Haskell Representation

The syntax is modelled following Definition 3.1.

```
type Agent = Int

data RegForm = Prop Proposition | Not RegForm | Disj RegForm RegForm | KHI Agent RegForm RegForm
    deriving (Eq, Show, Ord)
```

The automaton type implements Definition 3.2.

```
type Successors = [State]
type ActionAtState = (State, Action)

data Automaton = ATMN {
    statesA :: [State],
    actionsA :: [Action],
    transitionsA :: [(ActionAtState, Successors)],
    initial :: [State],
    final :: [State]
} deriving (Eq, Show, Ord)
```

The product digraph from Definition 3.3 is implemented below. The helper functions `getAutNext` and `getLtsNext` retrieve successor states in the automaton and LTS respectively. They are omitted here.

```
type GVertex = (State, State) -- (Automaton State, LTS State)
type GEdge = (GVertex, GVertex)

data Digraph = Digraph {
    vSet :: [GVertex],
    eSet :: [GEdge]
} deriving (Eq, Show, Ord)
```

The following functions implement the relational notions from Definition 3.4.

```
type PlanSet = [Plan]

-- R_pi(u) = union of R_sigma(u) for all sigma in pi
rPiU :: Relations -> State -> PlanSet -> [State]
```

```
-- R_pi(X) = union of R_pi(u) for all u in X
rPiX :: Relations -> [State] -> PlanSet -> [State]
```

```haskell
-- SE(sigma) = set of all states at which sigma is strongly executable
seSigma :: [State] -> Relations -> Plan -> [State]
```

```haskell
-- SE(pi) = intersection of SE(sigma) for all sigma in pi
sePi :: [State] -> Relations -> PlanSet -> [State]
```

The functions above are not used directly in the model-checking algorithm, but are included to align with the notions from the paper. For the same reason, we omit the implementation of definition 3.5. Because 3.4 and 3.5 are simply same concepts but in different perspectives.

Finally, the reg-LTS^U model implements Definition 3.6.

```haskell
type Uncertainty = [(Agent, [Automaton])] -- U_i = {A_1,...}, for each agent i

data RegLTSU = RegLTSU{
        statesM :: [State],
        relationsM :: Relations,
        uncertainty :: Uncertainty,
        valuationM :: Valuation -- use the Valuation from singleAgent
    } deriving (Eq, Show, Ord)

-- Given an agent i, return U_i = [A_1,...]
getAgentAuts :: RegLTSU -> Agent -> [Automaton]
getAgentAuts m agent =
    fromMaybe [] (lookup agent (uncertainty m))
```

3.3 Model checker in Haskell

We now discuss the implementation model checking algorithms for $reg\text{-}\mathcal{L}^U_{Kh}$ in Haskell.

Given a finite reg-LTS^U $\mathcal{S} = (S, (R_a)_{a \in Act}, (U_i)_{i \in Agt}, V)$, a state $s \in S$, and a formula $Kh_i(\varphi, \psi)$, we check whether $\mathcal{S}, s \models Kh_i(\varphi, \psi)$ by iterating over all automata $\mathcal{A} \in U_a$ and verifying two conditions:

1. $[\![\varphi]\!]^{\mathcal{S}} \subseteq \mathrm{SE}(L(\mathcal{A}))$
2. $R_{L(\mathcal{A})}([\![\varphi]\!]^{\mathcal{S}}) \subseteq [\![\psi]\!]^{\mathcal{S}}$

Once we find such automaton, the formula $Kh_i(\varphi, \psi)$ is then satisfied. Since $L(\mathcal{A})$ may be infinite, directly enumerating all plans is not feasible. Following [2], we handle each condition via a separate algorithm, both of which reduce the problem to reachability checks on finite graphs.

To perform the reachability checks in PTIME, we use the following *depth-first search* algorithm:

Require: Digraph $G = (V, E)$, start vertex v_0, target set $T_0 \subseteq V$
DFS($G, v_0, T_0, \emptyset$)
function DFS($G, v, T, visited$)
    if $v \in T$ **then return true**
    end if
    if $v \in visited$ **then return false**
    end if
    $visited \leftarrow visited \cup \{v\}$
    for $w \in \mathrm{successors}(G, v)$ **do**

```
        if DFS(G, w, T, visited) then return true
        end if
    end for
    return false
end function
```

The algorithm above runs in $O(|V| + |E|)$, which is polynomial. In Haskell:

```haskell
-- Successors of a vertex in the digraph
successors :: Digraph -> GVertex -> [GVertex]
successors g v = [v' | (u, v') <- eSet g, u == v]

-- DFS reachability: is any vertex in targetSet reachable from start?
-- The type can be read as: digraph -> start -> targets -> visited_set -> Bool
dfsReachable :: Digraph -> GVertex -> [GVertex] -> [GVertex] -> Bool
dfsReachable _ start targetSet _
    | start `elem` targetSet = True
dfsReachable _ start _ visited
    | start `elem` visited = False
dfsReachable g start targetSet visited =
    any (\v -> dfsReachable g v targetSet (start : visited)) (successors g start)
```

The algorithm that checks Condition (1) from [2] works as follows:

Require: reg-LTS^U $\mathcal{S}$, state $s \in S$, automaton $\mathcal{A} = (Q, Act, \delta, I, F)$

```
function CheckSE-Original(S, s, A)
    Build the product digraph G = (Q × S, E)
    for a ∈ Act do
        for t ∈ S such that R_a(t) = ∅ do
            for q ∈ Q such that δ(q, a) ≠ ∅ do
                for q_0 ∈ I do
                    if (q, t) is reachable from (q_0, s) in G then
                        return true
                    end if
                end for
            end for
        end for
    end for
    return false
end function
```

Instead of using the nested for-loops, we first collect all *bad vertices* via set comprehension, i.e.

$$Bad = \{(q, t) \in Q \times S \mid \exists\, a \in Act : \delta(q, a) \neq \emptyset \text{ and } R_a(t) = \emptyset\},$$

and then perform a single DFS reachability check from each initial vertex (q_0, s) to the set Bad. This is equivalent to the original procedure, but avoids repeated reachability checks.

A bad vertex (q, t) represents a situation where the automaton expects to perform some action a at state q, but the LTS cannot execute a at state t. If such a vertex is reachable, it means that some plan in $L(\mathcal{A})$ fails to be strongly executable.

Remark: This function might be more accurately called `checkNotSE` *rather than* `checkSE`*, since it returns true exactly when strong executability fails.*

Require: reg-LTS^U $\mathcal{S} = (S, (R_a), (U_i), V)$, state $s \in S$, automaton $\mathcal{A} = (Q, Act, \delta, I, F)$

```
function CheckSE(S, s, A)
    Build product digraph G = (Q × S, E)
```

$Bad \leftarrow \{(q,t) \in V \mid \exists\, a \in Act : \delta(q,a) \neq \emptyset \text{ and } R_a(t) = \emptyset\}$
for $q_0 \in I$ **do**
  if DFS((q_0, s)) with target Bad **then return true**
  end if
end for
return false
end function

Require: reg-LTS^U $\mathcal{S}$, automaton $\mathcal{A}$, truth set $[\![\varphi]\!]^{\mathcal{S}}$
function CheckCond1($\mathcal{S}, \mathcal{A}, [\![\varphi]\!]^{\mathcal{S}}$)
for $s \in [\![\varphi]\!]^{\mathcal{S}}$ **do**
  if CheckSE($\mathcal{S}, s, \mathcal{A}$) **then return false**
  end if
end for
return true
end function

In Haskell:

```haskell
-- Compute the set of bad vertices in G.
-- A vertex (q, t) is bad iff there exists an action a such that:
--   (i)  the automaton has a transition from q under a, and
--   (ii) the LTS has no successor from t under a.
badVertices :: RegLTSU -> Automaton -> [GVertex]
badVertices m atmn =
    nub [ (q, t)
        | q <- statesA atmn,
        t <- statesM m,
        a <- actionsA atmn,
        not (null (getAutNext atmn q a)),  --  delta(q,a) != empty
        null (getLtsNext m t a)            --  R_a(t) == empty
        ]

-- checker whether there is a path from (q0, s) to a bad vertex.
-- If so, return True. This means that s not in SE(L(A)).
-- Else, return False. This means that s in SE(L(A)).
checkSE :: RegLTSU -> State -> Automaton -> Bool
checkSE m s atmn =
    let g   = buildDigraph m atmn
        bad = badVertices m atmn
    in  any (\q0 -> dfsReachable g (q0, s) bad []) (initial atmn)

checkCond1 :: RegLTSU -> Automaton -> [State] -> Bool
checkCond1 m atmn =
    all (\s -> not (checkSE m s atmn))
```

Let $\mathcal{A}_1$ and $\mathcal{A}_2$ be two automata. To check whether $L(\mathcal{A}_1) \cap L(\mathcal{A}_2) \neq \emptyset$, it suffices to construct the product automaton and check whether any accepting state pair in $F_1 \times F_2$ is reachable from some initial state pair in $I_1 \times I_2$.

Require: Automata $\mathcal{A}_1 = (Q_1, Act_1, \delta_1, I_1, F_1)$ and $\mathcal{A}_2 = (Q_2, Act_2, \delta_2, I_2, F_2)$
function IntersectionNonEmpty($\mathcal{A}_1, \mathcal{A}_2$)
//Build product automaton
$V \leftarrow Q_1 \times Q_2$
$E \leftarrow \{((q_1, q_2), (q_1', q_2')) \mid a \in Act_1 \cap Act_2,\ q_1 \xrightarrow{a} q_1' \in \delta_1,\ q_2 \xrightarrow{a} q_2' \in \delta_2\}$
$G \leftarrow (V, E)$
$T \leftarrow F_1 \times F_2$
for $(q_1, q_2) \in I_1 \times I_2$ **do**
  if DFS((q_1, q_2)) with target T **then return true**
  end if

```
    end for
    return false
end function
```

To check Condition (2), it suffices to check whether

$$L(\mathcal{A}) \cap \bigcup \{L(\mathcal{A}_{(t_1,t_2)}) \mid t_1 \in [\![\varphi]\!]^{\mathcal{S}},\, t_2 \in [\![\neg\psi]\!]^{\mathcal{S}}\} = \emptyset,$$

where $\mathcal{A}_{(t_1,t_2)} = (S, Act, \delta', \{t_1\}, \{t_2\})$ is the path automaton whose transitions mirror the LTS, i.e. $\delta'(t, a) = \{t' \mid (t, t') \in R_a\}$. Its language is precisely $L(\mathcal{A}_{(t_1,t_2)}) = \{\sigma \in Act^* \mid t_2 \in R_\sigma(t_1)\}$.

By distributivity of $\cap$ over $\bigcup$, this is equivalent to checking that

$$L(\mathcal{A}) \cap L(\mathcal{A}_{(t_1,t_2)}) = \emptyset \quad \text{for all } (t_1, t_2) \in [\![\varphi]\!]^{\mathcal{S}} \times [\![\neg\psi]\!]^{\mathcal{S}}.$$

Require: reg-LTS^U $\mathcal{S}$, automaton $\mathcal{A}$, truth sets $[\![\varphi]\!]^{\mathcal{S}}$ and $[\![\neg\psi]\!]^{\mathcal{S}}$

```
function CheckCond2(S, A, [[φ]]^S, [[¬ψ]]^S)
    for t1 ∈ [[φ]]^S do
        for t2 ∈ [[¬ψ]]^S do
            if IntersectionNonEmpty(A, A_(t1,t2)) then return false
            end if
        end for
    end for
    return true
end function
```

In Haskell:

```haskell
-- Construct A_(t1, t2)
-- This automaton accepts all plans from t1 to t2
buildPathAutomaton :: RegLTSU -> State -> State -> Automaton
buildPathAutomaton m t1 t2 = ATMN
    { statesA      = statesM m,
    actionsA     = acts,
    transitionsA = [ ((s, a), getLtsNext m s a) | s <- statesM m, a <- acts],
    initial      = [t1],
    final        = [t2]
    }
  where
    -- Collect all actions that appear in the LTS relations
    acts = nub [a | (a, _) <- relationsM m]

-- Check the intersection of L(A1) and L(A2) is empty or not
intersectionNonEmpty :: Automaton -> Automaton -> Bool
intersectionNonEmpty a1 a2 =
    any (\sv -> dfsReachable prodGraph sv targets []) startVs
  where
    -- Shared actions
    sharedActs = nub [a | a <- actionsA a1, a `elem` actionsA a2]

    -- Product graph vertices and edges
    prodNodes = [(q1, q2) | q1 <- statesA a1, q2 <- statesA a2]
    prodEdges = [ ((q1, q2), (q1', q2'))
                | (q1, q2) <- prodNodes,
                a          <- sharedActs,
                q1'        <- getAutNext a1 q1 a,
                q2'        <- getAutNext a2 q2 a
                ]
    prodGraph = Digraph prodNodes prodEdges

    -- Start from any pair of initial states
    startVs = [(q1, q2) | q1 <- initial a1, q2 <- initial a2]
```

```
    -- Target: any pair of accepting states
    targets = [(q1, q2) | q1 <- final a1, q2 <- final a2]

-- Check for each A: for all (t1,t2): L(A) \cap L(A_(t1, t2)) = empty?
-- We check the emptiness of the intersection by checking the reachability of the product graph(automaton)
-- If empty, then cond 2 is SAT
checkCond2 :: RegLTSU -> Automaton -> [State] -> [State] -> Bool
checkCond2 m aut phiStates negPsiStates = not (any violates pairs)
  where
    pairs = [(t1, t2) | t1 <- phiStates, t2 <- negPsiStates]
    -- Check the intersection. Non-empty => cond 2 is UNSAT.
    violates (t1, t2) =
        let pathAut = buildPathAutomaton m t1 t2
        in  intersectionNonEmpty aut pathAut
```

Following the semantics defined in Definition 3.7 and putting things together, we complete the model checker for $reg\text{-}\mathcal{L}^{U}_{Kh}$.

```
-- [[phi]]= set of states that phi holds
truthSet :: RegLTSU -> RegForm -> [State]
truthSet m f = [s | s <- statesM m, isTrueReg (m, s) f]

-- Satisfaction relation for the propositional fragment
isTrueReg :: (RegLTSU, State) -> RegForm -> Bool
```

```
-- Kh_a(phi, psi) holds iff there exists A in U_a such that
-- (1) [[phi]] is subset of SE(L(A))
-- (2) R_{L(A)}([[phi]]) is subset of [[psi]]
isTrueReg (m, _) (KHI agent phi psi) =
    any (\aut -> checkCond1 m aut phiStates
              && checkCond2 m aut phiStates negPsiStates
    ) (getAgentAuts m agent)
  where
    phiStates    = truthSet m phi          -- [[phi]]
    negPsiStates = truthSet m (Not psi)    -- [[neg psi]]

-- Infix alias for the satisfaction relation
(||=) :: (RegLTSU, State) -> RegForm -> Bool
(||=) = isTrueReg
```

3.4 Parsing for $reg\text{-}\mathcal{L}^{U}_{Kh}$

Formulas may be created in `ghci` using `parseRegForm`. We omit most of the code here. The following inputs are accepted.

- `p` or `P` followed by an integer n returns `Prop n`
- `!p` returns `Not p`
- `p v q` or `p V q` returns `Disj p q`
- `KHi p q` returns `KHI i p q`, where `i` is an agent index
- `p -> q` returns `Disj (Not p) q` (as an abbreviation)
- `p & q` returns `Not (Disj (Not p) (Not q))` (as an abbreviation)

```
parseRegForm :: String -> Either ParseError RegForm
parseRegForm = parse (pRegForm <* eof) "input"

evalRegForm :: (RegLTSU, State) -> String -> Bool
evalRegForm (m, s) str =
    case parseRegForm str of
        Right f -> isTrueReg (m, s) f
        Left _  -> error "Invalid formula"
```

3.5 Random Generation for $reg\text{-}\mathcal{L}^U_{Kh}$

3.5.1 Model Generation with Parameters

To facilitate testing in the multi-agent setting, we provide a function that generates a random reg-LTS^U model with a fixed number of states, propositions, actions, and agents. We omit the code here.

The generated model includes:

- a finite set of states $\{1, \ldots, n\}$,
- a set of actions $\{1, \ldots, k\}$,
- a valuation assigning each state a random subset of propositions $\{1, \ldots, m\}$,
- a family of action-labelled transitions,
- and for each agent, a set of automata representing uncertainty over plans.

In addition, we implement a sanity check on the uncertainty component. This verify that the languages recognized by distinct automata for the same agent are pairwise disjoint.

We do a small example here to generate models and test formulas interactively using `ghci`:

```
ghci> m <- generate (generateRegLTSU 3 2 1 1)

ghci> print m
RegLTSU {statesM = [1,2,3],
relationsM = [(1,[(1,3),(3,2),(2,2),(2,1)])],
uncertainty = [(1,[
    ATMN {statesA = [1,2,3], actionsA = [1],
    transitionsA = [((1,1),[]),((2,1),[1,3]),((3,1),[2])],
    initial = [1,3], final = [1]}])],
    valuationM = [(1,[]),(2,[1]),(3,[])]}

ghci> sanityCheckRegLTSU m
True

ghci> evalRegForm (m, 1) "KH1 p1 p2"
False
```

3.5.2 Arbitrary Instances for QuickCheck

To support property-based testing, we define generators for both formulas and models in the multi-agent setting.

Since the structure of formulas is independent of any particular model, the generator is defined separately from model generation. However, to avoid mismatches between formulas and models, we parameterize the generator by the number of agents. This ensures that all occurrences of the modality Kh_i refer only to valid agent indices.

```
-- Generate a random RegForm with bounded size and a fixed number of agents
generateRegForm :: Int -> Int -> Gen RegForm
generateRegForm numAgents = randomRegForm
  where
    numAgents' = max 1 numAgents

    randomRegForm :: Int -> Gen RegForm
    randomRegForm 0 = Prop <$> choose (1, 5)
    randomRegForm n = oneof
        [ Prop <$> choose (1, 5)
        , Not <$> randomRegForm (n - 1)
        , Disj <$> randomRegForm (n `div` 2) <*> randomRegForm (n `div` 2)
        , KHI <$> choose (1, numAgents')
              <*> randomRegForm (n `div` 2)
              <*> randomRegForm (n `div` 2)
        ]
```

4 Testing and Validation

We now use the library `QuickCheck` to randomly generate models and states in order to test several basic semantic and implementation-level properties of both the basic single-agent logic $\mathcal{L}_{Kh}$ and the multi-agent regular plan logic reg-$\mathcal{L}^U_{Kh}$.

To perform these tests, we first define auxiliary types `PointedModel` (for single-agent models) and `PointedRegModel` (for multi-agent models), together with their `Arbitrary` instances. This allows `QuickCheck` to choose a random evaluation point inside a randomly generated model.

4.1 Tests for $\mathcal{L}_{Kh}$

We begin by testing several basic properties of the single-agent logic.

The first group of tests concerns the Boolean fragment:

- Double Negation Elimination: $\neg\neg\varphi \equiv \varphi$
- Commutativity of Conjunction: $\varphi \wedge \psi \equiv \psi \wedge \varphi$
- Identity of Conjunction: $\varphi \wedge \top \equiv \varphi$

We then test a property specific to the Kh-modality. In our semantics, $Kh(\psi, \varphi)$ is *global*: its truth value does not depend on the current evaluation state, since its semantic clause quantifies

over all states satisfying the precondition ψ. Hence, if $Kh(\psi, \varphi)$ holds at one state of a model, it should hold at every state of that model.

Finally, we test two basic implementation-level properties concerning plans: executing the empty plan should leave the current state unchanged, and the empty plan should be strongly executable at every state.

4.2 Tests for $reg\text{-}\mathcal{L}^{U}_{Kh}$

In the multi-agent setting, we test several basic semantic and implementation-level properties of the Kh_i-modality.

First, we test the case of an impossible precondition. If the precondition is contradictory, then no states satisfy it, and both semantic conditions of $Kh_i(\psi, \varphi)$ are trivially satisfied. Hence, $Kh_i(\psi, \varphi)$ should hold vacuously in this case.

Second, as in the single-agent case, we verify that $Kh_i(\psi, \varphi)$ is global in the model: its truth value does not depend on the evaluation state, since its semantics quantifies over all states satisfying the precondition.

Finally, we include two implementation-level checks related to the automata-based part of the model checker. We first verify that the procedure `intersectionNonEmpty` is symmetric in its two automaton arguments. We also test that the path automata constructed in the implementation faithfully mirror the transition structure of the underlying LTS.

Running `stack test` in the terminal yields the following relevant test output.

```
KHora> test (suite: quickcheck-logic)

Single-Agent Logic L_Kh
  Boolean: Double Negation Elimination (!!f <=> f)
    +++ OK, passed 100 tests.
  Boolean: Commutativity of Conjunction (f1 ^ f2 <=> f2 ^ f1)
    +++ OK, passed 100 tests.
  Boolean: Identity of Conjunction (f ^ T <=> f)
    +++ OK, passed 100 tests.
  Kh is global: truth does not vary between states
    +++ OK, passed 100 tests.
  Empty plan leaves the current state unchanged
    +++ OK, passed 100 tests.
  Empty plan is strongly executable at every state
    +++ OK, passed 100 tests.
Multi-Agent Logic reg-L^U_KH
  Vacuous Precondition: knowing how from contradiction holds vacuously
    +++ OK, passed 100 tests.
  Kh_i is global: agent ability is a model-wide property
    +++ OK, passed 100 tests.
  Intersection checking is symmetric:
  the intersection of L(A1) and L(A2) is non-empty
```

```
  iff the intersection of L(A2) and L(A1) is non-empty
    +++ OK, passed 100 tests.
  Path automata mirror the LTS transitions
    +++ OK, passed 100 tests.

Finished in 1.1135 seconds
10 examples, 0 failures

KHora> Test suite quickcheck-logic passed
Completed 2 action(s).
```

Overall, these results provide evidence that the implementation correctly captures the intended semantic and implementation-level properties of the logics.

5 Web Interface

In this section we introduce a web-application to demo our code. The web-app is consistent of a single page allowing users to do the following:

- Parse single or multi agent formulas to our Haskell syntax
- Generate random single agent formulas in our Haskell syntax
- Generate single or multi agent models in our Haskell syntax using size-parameters
- Generate random single agent models in our Haskell syntax

The app also provides instructions on how to model-check.

Providing a user-interface (UI) is an additional feature to our project. Therefore we have kept our implementation extremely simple. We use a simple lightweight RESTful framework called scotty, and plain HTML and CSS is used for layout and styling. We serve a single plain JavaScript script to connect the UI to our Haskell code. We omit most of the code here.

The web-app is currently not deployed to the web. However, it is possible to run the app by running `stack build` and `stack run`.

```haskell
main :: IO ()
main = scotty 3000 $ do
  middleware $ staticPolicy (noDots >-> addBase "static")

  get "/" $ do
    file "static/index.html"
```

6 Conclusion

In this work, we presented a Haskell implementations of two knowing-how logics: the basic logic $\mathcal{L}_{Kh}$ and its extension with regularity constraints reg-$\mathcal{L}^{U}_{Kh}$. For $\mathcal{L}_{Kh}$, we implemented an

explicit model checker based on bounded plan search. For $reg\text{-}\mathcal{L}^{U}_{Kh}$, we implemented a model checker following the procedure described in [2]. Since the original paper does not spell out all implementation details of the reachability component, we made these constructions explicit by using depth-first search. We also incorporated a small optimization in the implementation of `CheckSE`: instead of performing reachability tests inside nested loops, we first compute the set of bad points and then perform the reachability checks, thereby avoiding repeated reachability computations. To support experimentation, we provided parsers for both logics, generators for random models and formulas, a collection of `QuickCheck` and `HSpec` tests, and a lightweight web interface.

For future work, we plan to extend the checker for the basic knowing-how logic to a full decision procedure, and broaden the system to support additional logics.

References

[1] C. Areces, R. Fervari, A. R. Saravia, and F. R. Velázquez-Quesada. Uncertainty-based semantics for multi-agent knowing how logics. In *Proceedings of the 18th Conference on Theoretical Aspects of Rationality and Knowledge (TARK 2021)*, volume 335 of *EPTCS*, pages 23–37, 2021.

[2] S. Demri and R. Fervari. Model-checking for ability-based logics with constrained plans. In *Proceedings of the 37th AAAI Conference on Artificial Intelligence (AAAI-23)*, volume 37, pages 6305–6312, 2023.

[3] Y. Wang. A logic of knowing how. In *Logical Reasoning and Interaction (LORI'15)*, volume 9394 of *Lecture Notes in Computer Science (LNCS)*, pages 392–405. Springer, 2015.